Move a historical Solaris installation to a maintained platform

Cacti 1.3 development series

Documentation source base: d8a327c12a91
Built: 2026-10-27

This snapshot follows the development documentation. The accompanying build manifest records each source file hash.

For Cacti 1.2, use the maintenance manual for your installed release.

Video thumbnails are represented by links; local screenshots are embedded.

Copyright (c) 2004-2026 The Cacti Group

Contents

1. Move a historical Solaris installation to a maintained platform 3
2. Requirements 5
3. General Installing Instructions 8
4. How to Graph Your Network 15
5. Data Source Debugging 18
6. Debugging 21
7. Hardening a Cacti Installation 24
8. Spine 29
9. Upgrading Cacti Under UNIX Compatible Operating Systems 32
10. Upgrading Cacti Under FreeBSD 35

Move a historical Solaris installation to a maintained platform

This guide replaces the archived Solaris installation procedure with a migration procedure. It does not certify a new Solaris installation of Cacti 1.3. Solaris is not among the platforms with maintained installation guidance in this manual. Choose a target from Requirements. For released Cacti 1.2, use the 1.2.x manual; this documentation otherwise follows the unreleased Cacti 1.3 development branch.

Inventory the source

Record the exact Cacti, PHP, database, RRDtool, Spine, and plugin versions; collector IDs and hostnames; database encoding; configured paths; schedules; and the poller and web identities. Include scripts, query definitions, templates, credentials, certificates, and RRD files outside the application folder. Keep credentials out of screenshots and support messages.

Take database and filesystem backups before any change. Test restoring them in an isolated environment. An old Cacti database may need intermediate upgrades: follow the version-specific release notes and upgrade instructions for each required step rather than importing an unknown historical schema into the newest application. Never import a new-install `cacti.sql` over the preserved database.

Prepare an isolated target

Install a maintained Linux or FreeBSD system with dependencies compatible with the chosen Cacti release. Follow the platform's installation guide and general instructions. Keep its poller schedule disabled and restrict web access while restoring data.

Restore copies of the database, configuration, and required assets. Adjust absolute paths, service identities, and permissions to the target. Reinstall compatible plugins and dependencies from trusted sources. Keep application code protected from runtime write access. Complete upgrades once, in the sequence required by the selected releases, against the isolated copy.

Transfer RRD data between architectures

Do not assume binary RRD files are portable between Solaris and the new host. Use RRDtool's dump on the source and restore on the target when the binary format or architecture differs. For one example file:

```
# Source host, after pausing writes:
rrdtool dump /source/rra/example.rrd > /backup/example.xml
# Target host; choose a new destination that does not already exist:
rrdtool restore /backup/example.xml /target/rra/example.rrd
rrdtool info /target/rra/example.rrd
rrdtool lastupdate /target/rra/example.rrd
```

Replace every example path. Test conversion on copies with the RRDtool versions actually installed. Preserve filenames and Cacti's association with each RRD. Compare data source definitions, last timestamps, and representative historical graphs. XML dumps can be large; allow enough storage for backups and conversion.

Validate, cut over, and retain rollback

Check web and CLI PHP, SNMP access from the new collector, database connectivity, permissions, graph history, and data debugging. Do not poll production devices from both installations during validation.

Schedule a maintenance window. Stop source polling and verify that no poller is running. Take the final backup and transfer changed data. Enable exactly one target schedule only after validation. Verify new samples arrive and historical graphs remain readable. Update DNS, access controls, backups, and monitoring deliberately; apply hardening.

Retain the stopped source and backups for rollback. If rollback is needed, stop target polling first. Decide how to preserve any samples collected after cutover before restoring the source. Never run both schedules to fill gaps.

Copyright (c) 2004-2026 The Cacti Group

Requirements

This manual follows the unreleased Cacti 1.3 `develop` branch. Use the 1.2.x manual for released Cacti 1.2 installations. Verify the requirements of the exact revision or release you deploy; development requirements can change.

Supported operating systems

Platform guidance checked on 26 October 2026. Use the current security-updated point release in a vendor-supported stable branch. The examples below target these maintained operating systems; vendor OS support does not certify every Cacti, PHP, database, or third-party binary combination.

Operating system	Versions used for installation guidance	Lifecycle source
RHEL	Current supported minor release of 9 or 10	Red Hat lifecycle
Rocky Linux	Current minor release of 9 or 10	Rocky releases
AlmaLinux	Current minor release of 9 or 10	AlmaLinux releases
Ubuntu Server	24.04 LTS, 26.04 LTS, or 26.10	Ubuntu lifecycle
Debian	13 stable; 12 only with applicable Debian LTS coverage	Debian lifecycle
FreeBSD	14.5-RELEASE or 15.1-RELEASE, with current patches	FreeBSD security support
Windows Server	2022 or 2025, with current updates	Server 2022, Server 2025

Check the linked lifecycle information before deployment. LTS coverage can exclude architectures or packages. Use a maintained PHP and database version that also meets this Cacti branch's requirements. A supported OS with an incompatible default PHP version is not ready for installation. Do not use a beta OS or an expired point release for these procedures.

Software Requirements

Component	Development requirement or recommendation
Web server	PHP-capable Apache, nginx, or IIS FastCGI
PHP	PHP 8.3 or later in the PHP 8 series (`^8.3` in Composer)
Database	MySQL 8.4+ or MariaDB 10.11+ recommended by the installer
RRDtool	Version 1.5 or later
SNMP	Net-SNMP command-line tools for operations using external binaries
Spine	Optional; use a version compatible with Cacti and its runtime libraries

The PHP requirement comes from the dependency manifest and runtime minimum. The database values are the branch's installer recommendations, rather than a certification of every vendor release. Check the actual server and extensions before installation.

PHP extensions

Cacti checks the CLI and web PHP environments separately. Install the extensions required by the selected revision's Composer manifest and lock file, and satisfy the installer checks. The combined requirements include:

- `ctype`, `curl`, `date`, `dom`, `filter`, `gd`, `gmp`, `hash`, `intl`, `json`, `ldap`, `mbstring`, `mysqlnd`, `openssl`, `pcre`, `PDO`, `pdo_mysql`, `Phar`, `session`, `simplexml`, `sockets`, `spl`, `sqlite3`, `standard`, `xml`, and `zlib`.
- `posix` and CLI `pcntl` on Unix.
- `com_dotnet` for the Windows installer checks.

Some modules are built into PHP. Check `php -v`, `php --ini`, and `php -m` for the CLI, then check web PHP through the installer. Enabling an extension in one PHP configuration does not enable it in the other.

The PHP SNMP extension is optional. External Net-SNMP tools are still needed for collection paths such as some SNMPv3 and bulk requests. Configure their paths and verify execution under the actual web and collector accounts.

Composer and source deployments

A source checkout or GitHub source archive needs its locked PHP dependencies. Run these commands from the Cacti source root as the deployment owner:

```
composer install --no-dev
composer check-platform-reqs --no-dev
```

Use `composer install`, which reads `composer.lock`. Do not run `composer update` during installation or upgrade. A complete release bundle can already contain `include/vendor/autoload.php`; verify its generated platform checks and the release's instructions rather than assuming every archive contains vendor files.

The current manifest requires `ext-pcntl` and `ext-posix` unconditionally. Native Windows PHP cannot supply these modules. Source dependency installation is therefore blocked on native Windows, even though the installer has Windows-specific checks. A bundled artifact must also pass its runtime platform check. See Installing Under Windows before choosing a build. Do not bypass platform checks to force installation.

Hardware Sizing

Size from the polling interval, number of data sources, retention profiles, concurrent graph requests, collector settings, and measured storage latency. Device count alone is not a reliable sizing rule. Test representative polling and graphing load before increasing the deployment.

Use storage that can sustain RRD updates and database writes within the polling window. Measure CPU, memory, I/O latency, database connections, and poll completion time. Remote collectors can distribute collection work; they also require connectivity, synchronization, and operational monitoring. Spine process and thread counts should be tuned against those measurements.

Database configuration

Enable timezone support and grant the application account `SELECT` access to `mysql.time_zone_name`. The grant does not populate that table. Import the vendor's timezone data using the procedure for your operating system.

Use `utf8mb4` and the `utf8mb4_unicode_ci` collation consistently for the Cacti database. Changing server defaults does not convert existing tables; back up and plan any conversion of an installed database separately.

Start with the database vendor's defaults and review the installer's checks for the installed engine and version. Do not copy a fixed `my.cnf` block from another server. Size global buffers and per-connection buffers together with PHP and operating-system memory. Estimate Spine's connection demand as:

```
total_connections = total_processes * (total_threads + script_servers + 1)
```

Allow additional connections for web requests, plugins, and maintenance. Inspect actual storage engines before applying memory-table sizing guidance.

Keep InnoDB doublewrite protection enabled. Disabling it removes protection against incomplete page writes; an SSD alone does not establish atomic page writes. Review the durability consequences before changing commit flushing or other crash-recovery settings.

Database variables differ by engine and release. Removed settings can prevent startup. For example, MySQL 8 no longer accepts `innodb_file_format`, `innodb_large_prefix`, or `innodb_additional_mem_pool_size`, and MariaDB 10.6 removed `innodb_buffer_pool_instances`. Validate settings against the installed server's documentation and Cacti's version-aware checks before restarting. See the MySQL reference or MariaDB reference.

Accounts and platform controls

Keep application code owned by the administrator or deployment account. Give web and collector identities the read, execute, and runtime-directory write access they need. Grant temporary installation or import access only to the required paths, and remove it afterward. Keep database credentials, CSRF secrets, private caches, RRD files, and backups inaccessible over HTTP.

Configure SELinux, AppArmor, container permissions, or IIS request filtering for the required operations. Do not remove confinement globally to make installation checks pass. Follow the platform installation guide and Hardening a Cacti Installation.

Copyright (c) 2004-2026 The Cacti Group

General Installing Instructions

Use only the supported operating systems and maintained dependencies compatible with this Cacti branch.

Install the packages required by the selected operating system and verify that the configured web server, PHP handler, and database start at boot.

Containers

Use an unprivileged container with its normal security profile where possible. Diagnose the specific ICMP, socket, or filesystem permission that fails and adjust only the required capability or policy. Do not remove AppArmor confinement or make the entire container privileged as a general prerequisite.

Required Packages for Most Operating Systems

Depending on your operating system and PHP version, certain packages are required for Cacti. The largest variable in these requirements come with regard to PHP and MySQL/MariaDB.

Installation requirements include the packages below. The installation of these packages will vary by operating system.

Base OS

- apache, IIS, or nginx
- net-snmp, net-snmp-utils
- rrdtool
- help2man (for spine)
- dos2unix (for spine)
- development packages (gcc, automake, autoconf, libtool, either mysql-devel or mariadb-devel, net-snmp-devel, help2man)

  (for spine)

Database

Use the database versions and configuration described in Requirements. The development branch recommends MySQL 8.4+ or MariaDB 10.11+. Check the installed server with the Cacti installer.

- mysql
- mysql-server
- libmysqlclient

or

- mariadb
- mariadb-server
- libmariadbclient

PHP Modules

Use PHP 8.3 or later in the PHP 8 series and the extensions listed in Requirements. Verify the CLI with `php -v` and `php -m` and check web PHP separately in the installer. Source deployments also need Composer and all extensions required by the selected revision's lock file. Run `composer install --no-dev` as the deployment owner, then `composer check-platform-reqs --no-dev`. Use the lock file rather than updating dependencies during installation.

Systems using PHP-FPM

After changing PHP configuration, test and restart the actual FPM service. Service names can be versioned, for example on Debian and Ubuntu. Check the installed unit rather than assuming that `php-fpm` is the service name. Confirm its socket path and permissions match the web-server handler.

FreeBSD

Use a patched release listed in supported operating systems. Choose binary packages or the Ports Collection using the FreeBSD Handbook. Run package and service administration commands as root. Use the dedicated poller account for collection.

Choose the Cacti version and PHP flavor

The FreeBSD `net-mgmt/cacti` port currently packages Cacti 1.2.31, not the unreleased Cacti 1.3 development branch covered by this manual. For a packaged release, follow its installed instructions and the 1.2.x manual. For development source, meet Requirements and follow the source-install steps below using FreeBSD paths and service configuration. Do not overlay a source checkout onto files owned by `pkg`.

Cacti packages have PHP flavors. Discover the names available in your configured repository before selecting one:

```
pkg update
pkg search -S name -x '^cacti(-php[0-9]+)?$'
```

For example, if the repository offers `cacti-php84` and the installation uses PHP 8.4, review and install that flavor:

```
pkg install -n cacti-php84
pkg install cacti-php84
pkg info -D cacti-php84
pkg info -l cacti-php84
```

Replace the example name with the selected flavor. Read the transaction before accepting it, particularly changes to PHP extensions and database clients. Dependencies do not configure a web server or create a database server. Set up a compatible local MariaDB/MySQL service, or an existing remote database, and configure Apache or nginx with the matching PHP handler separately. Use FreeBSD `service` and `sysrc` for installed rc.d services; Linux `systemctl` examples do not apply.

Build from ports when custom options are required

Use a maintained ports tree and read `/usr/ports/UPDATING`. Clone a tree only if `/usr/ports` does not already contain one:

```
pkg install git
# New tree only:
git clone --depth 1 https://git.FreeBSD.org/ports.git /usr/ports
# Existing Git tree instead:
git -C /usr/ports pull --ff-only
```

Build the chosen flavor, for example:

```
cd /usr/ports/net-mgmt/cacti
make FLAVOR=php84 config
make FLAVOR=php84 install clean
```

The supported flavors and dependencies come from that ports revision. Follow the handbook's dependency-management guidance before mixing custom ports with binary packages. Rebuilding a port does not turn the packaged release into Cacti development source.

Check installed paths and services

The Cacti port uses these default locations. Custom ports options and existing Cacti settings can change them; inspect the installed files and application configuration before copying commands.

Item	Default location or account
Application and bundled schema	`/usr/local/share/cacti`
Database configuration	`/usr/local/share/cacti/include/config.php`
PHP configuration	`/usr/local/etc/php.ini`
RRD data	`/var/db/cacti/rra`
Log directory	`/var/log/cacti`
PHP CLI	`/usr/local/bin/php`
Packaged poller account	`cacti`
Web-server account	Commonly `www`; verify the service configuration

The port supplies `include/config.php.sample`; preserve an existing `include/config.php` and use the sample when creating a new one. Set the intended timezone in both CLI and web PHP. Initialize a new database from the bundled `cacti.sql`, grant the application account access to its database and `mysql.time_zone_name`, and verify database timezone data is loaded. Never import `cacti.sql` over an existing installation.

Check package-specific patches as well as upstream instructions. For example, the current port sets `$php_snmp_support = false` for new configurations; verify the installed setting and Net-SNMP paths when troubleshooting polling.

Configure write access only for the runtime directories needed by the actual poller and web processes. Keep application code and database credentials protected. Preserve Cacti's web-server restrictions on configuration, CLI, resource, and data directories.

Create one Cacti schedule. For a five-minute launcher, an example entry in the `cacti` user's crontab is:

```
*/5 * * * * /usr/local/bin/php -q /usr/local/share/cacti/poller.php
```

Match this cadence to **Cron/Daemon Interval** in Cacti. A system crontab needs the account field as well. Check for an existing cron entry or daemon first; do not run both. Use Data Source Debugging and the Cacti log to verify collection after completing the web installer.

Optional Spine poller

The Spine port is `net-mgmt/cacti-spine`; its package is `spine`. Install it only when its version matches the Cacti release you use:

```
pkg install -n spine
pkg install spine
```

Its default binary is `/usr/local/bin/spine`, with `/usr/local/etc/spine.conf.sample` as the configuration example. Supply the correct database connection in `spine.conf` and select the binary in Cacti. Verify the poller account can run it. Grant ICMP privileges only when the selected polling method requires them; do not apply a blanket setuid command.

For an existing installation, follow Upgrading Under FreeBSD.

Configure PHP

Verify that the modules are installed and configured correctly. There are several ways to do so - consult PHP configuration instructions for a complete description.

It is imperative that you set the `date.timezone` in your `/etc/php.ini`, or `/etc/phpX/apache/php.ini` and `/etc/phpX/cli/php.ini` files. Failure to do so will result in errors after the install is complete.

Most other PHP configuration is done automatically by the base OS, so there is no need to discuss that here.

Configure the Webserver (Apache)

Use the distribution's Apache/PHP integration, selecting either its PHP-FPM handler or a compatible Apache PHP module. Do not configure both handlers for the same files. The Enterprise Linux LAMP guide and Ubuntu/Debian guide provide examples.

Verify Apache's configuration before reloading. Restrict access during setup, use HTTPS, and ensure that PHP requests execute rather than expose source. Keep Cacti's shipped `.htaccess` restrictions active, or reproduce them in the server configuration if overrides are disabled. nginx and IIS need their own restrictions; they do not interpret Apache `.htaccess` files.

Configure MySQL/MariaDB

Initialize and secure the database using the procedure for the installed MySQL or MariaDB package. Use the administrator authentication method configured by that package; some Unix packages use local socket authentication. Store credentials securely and do not put passwords directly on shell command lines.

You must also load timezone information into the database. This is required for various plugin use. Later, you will be required to grant access to the `time_zone_name` table during the final installation steps.

```
mysql_tzinfo_to_sql /usr/share/zoneinfo | mysql -u root -p mysql
```

Since Cacti 1.x is supporting internationalization (i18n), it is important that the default character set for MySQL/MariaDB be i18n compatible. The Cacti installer will make specific recommendations on MySQL/MariaDB settings. Follow those as applicable for your OS.

Galera clustering: There are several tables which are set to use the MEMORY storage engine which do not get replicated among nodes which can cause problems. If you configure Cacti to only connect to one node of your cluster and are not load balancing this does not apply to you.

If you are running multiple nodes in a load-balanced environment where you connect to a VIP you should remove all but one node from rotation during Cacti installation or update. After the installation/update login to your MySQL server and execute the following commands to update those tables to use the InnoDB engine:

```
MariaDB [(none)]> use cacti;
MariaDB [cacti]> ALTER TABLE `automation_ips` ENGINE=InnoDB;
MariaDB [cacti]> ALTER TABLE `automation_processes` ENGINE=InnoDB;
MariaDB [cacti]> ALTER TABLE `data_source_stats_hourly_cache` ENGINE=InnoDB;
MariaDB [cacti]> ALTER TABLE `data_source_stats_hourly_last` ENGINE=InnoDB;
```

```
MariaDB [cacti]> ALTER TABLE `poller_output` ENGINE=InnoDB;
MariaDB [cacti]> ALTER TABLE `poller_output_boost_processes` ENGINE=InnoDB;
```

Before restoring database load balancing, verify the table-engine changes, replication state and successful polling on the configured topology. A fixed number of polling cycles is not proof that the cluster is ready.

Install and Configure Cacti

These are fresh source-install examples for Cacti development (1.3). Package-managed installations should use the maintainer's procedure instead. For an existing installation, follow Upgrading Cacti. The examples use `/var/www/html/cacti`; adjust that path and the service accounts for your OS. Use administrator privileges for software installation and permission changes.

Extract the source and create the database

Obtain the intended source revision in a new installation directory. Do not run these commands over an existing deployment:

```
git clone --branch develop --depth 1 https://github.com/Cacti/cacti.git cacti
mv cacti /var/www/html/cacti
cd /var/www/html/cacti
composer install --no-dev
composer check-platform-reqs --no-dev
```

The installation root should now contain `cacti.sql`, `poller.php` and `include/config.php.dist`, rather than another nested release directory.

Connect with a database administrator account:

```
mysql --user=root --password
```

Create the database and a dedicated application account. Replace the password placeholder before running this SQL, and use the same secret in the Cacti configuration. The database account is separate from the operating-system account that runs the poller.

```
CREATE DATABASE cacti CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'cactiuser'@'localhost'
  IDENTIFIED BY 'REPLACE_WITH_UNIQUE_DATABASE_PASSWORD';
GRANT ALL PRIVILEGES ON cacti.* TO 'cactiuser'@'localhost';
GRANT SELECT ON mysql.time_zone_name TO 'cactiuser'@'localhost';
EXIT;
```

Use an authorized database administrator to grant access. Do not edit `mysql.tables_priv` as a substitute for an authorized `GRANT` operation. Ensure the server's timezone tables are populated using the procedure for the installed MySQL or MariaDB server; granting access does not populate them.

Import the schema into the new database using the appropriate client:

```
mysql --user=root --password --default-character-set=utf8mb4 cacti \
  < /var/www/html/cacti/cacti.sql
```

The `--password` option prompts for the password. Use `mariadb` in place of `mysql` where appropriate for the installed server.

Configure access and complete installation

Copy the supplied configuration example:

```
cp /var/www/html/cacti/include/config.php.dist \
  /var/www/html/cacti/include/config.php
```

Edit the copied file to match the account created above:

```
$database_type     = 'mysql';
$database_default  = 'cacti';
$database_hostname = 'localhost';
$database_username = 'cactiuser';
$database_password = 'REPLACE_WITH_UNIQUE_DATABASE_PASSWORD';
$database_port     = '3306';
$url_path          = '/cacti/';
```

Identify the actual web and poller service accounts. Give them the required read/execute access to application files and external tools, and write access to the runtime paths they use, including RRD, log and cache paths. They need not be the same account. Do not assume a database username is also a local OS account or make the whole application tree writable.

During installation and package imports, grant the performing identity the additional access required by the installer, including query/script resources and the location used to initialize the CSRF secret. Remove temporary setup permissions afterward. Preserve read access to the generated secret; do not create an empty secret file. See the platform-specific installation guides for their account and permission examples.

Restrict installer access to the administrator and use HTTPS or a protected local connection before entering credentials. A freshly imported development schema uses application username `admin` and initial password `admin`. Open the configured Cacti URL, sign in and choose a unique administrator password when prompted. These are separate from the database credentials. Complete the installer's dependency, database, path and permission checks before allowing general access.

Schedule collection after installation

Use either cron or `cactid.service`, not both. The following examples assume an existing local user and group named `cacti` with the required polling permissions; replace them with the actual service identity for your system. Do not start polling before completing the web installer.

For a system cron file such as `/etc/cron.d/cacti`, use an absolute PHP CLI path and include the OS username:

```
*/5 * * * * cacti /usr/bin/php /var/www/html/cacti/poller.php >/dev/null 2>&1
```

The five-minute launch interval must match **Cron/Daemon Interval** under the poller settings. This is separate from individual data-source polling intervals. The redirection above works with `/bin/sh`; `&>` is Bash-specific.

The supplied systemd unit uses `Type=forking`. Confirm that the selected PHP CLI provides `pcntl_fork` before using it; otherwise use cron or install the appropriate PCNTL support. For the example CLI path, this should report `bool(true)`:

```
/usr/bin/php -r 'var_dump(function_exists("pcntl_fork"));'
```

For systemd, copy the supplied unit and edit the installed copy:

```
cp /var/www/html/cacti/service/cactid.service \
  /etc/systemd/system/cactid.service
```

Replace the unit's existing `User`, `Group`, `ExecStart` and `EnvironmentFile` settings with values for the actual deployment, retaining its other settings. For example:

```
User=cacti
Group=cacti
ExecStart=/usr/bin/php /var/www/html/cacti/cactid.php
EnvironmentFile=-/etc/default/cactid
```

The `-` allows the optional environment file to be absent. Disable the Cacti-specific cron entry before enabling the daemon, then load the edited unit:

```
systemctl daemon-reload
systemctl enable cactid.service
systemctl start cactid.service
systemctl status cactid.service
```

The unit manages the local daemon; it does not establish high availability. Verify scheduler execution, collector status, logs and recent graph data under the configured account.

(Optional) Install and Configure Spine

Spine is a very fast data collection engine, written in C. It is an optional replacement for cmd.php. If you decide to use it, you will have to install it explicitly. It does not come with cacti itself.

The easiest way is to install Spine using rpm or ports. You will find packages for Spine at the main cacti site or from your distribution.

To compile Spine, download it to any location of your liking. Then, issue from the downloaded directory following commands

```
./bootstrap
```

After `bootstrap` succeeds, follow the release's build instructions to configure, compile and install Spine.

Assuming, you've managed to install spine correctly, you will have to configure it. Use the installed `spine.conf.dist` as the example, and configure the actual Spine configuration path in Cacti. Use the same database account and secret as the Cacti configuration, and restrict file access to the poller account and administrators.

```
DB_Host     localhost
DB_Database cacti
DB_User     cactiuser
DB_Pass     REPLACE_WITH_UNIQUE_DATABASE_PASSWORD
DB_Port     3306
```

Considerations when using Proxies in front of Cacti (Cacti 1.2.23+)

A header allowlist does not authenticate a proxy. Trust forwarded client-IP headers only when a trusted proxy replaces those headers and untrusted clients cannot bypass it to reach Cacti directly. Leave forwarding disabled otherwise. In `include/config.php`, select only the headers your proxy sets. For a deployment using `X-Forwarded-For`, the setting is:

```
$proxy_headers = ['X-Forwarded-For'];
```

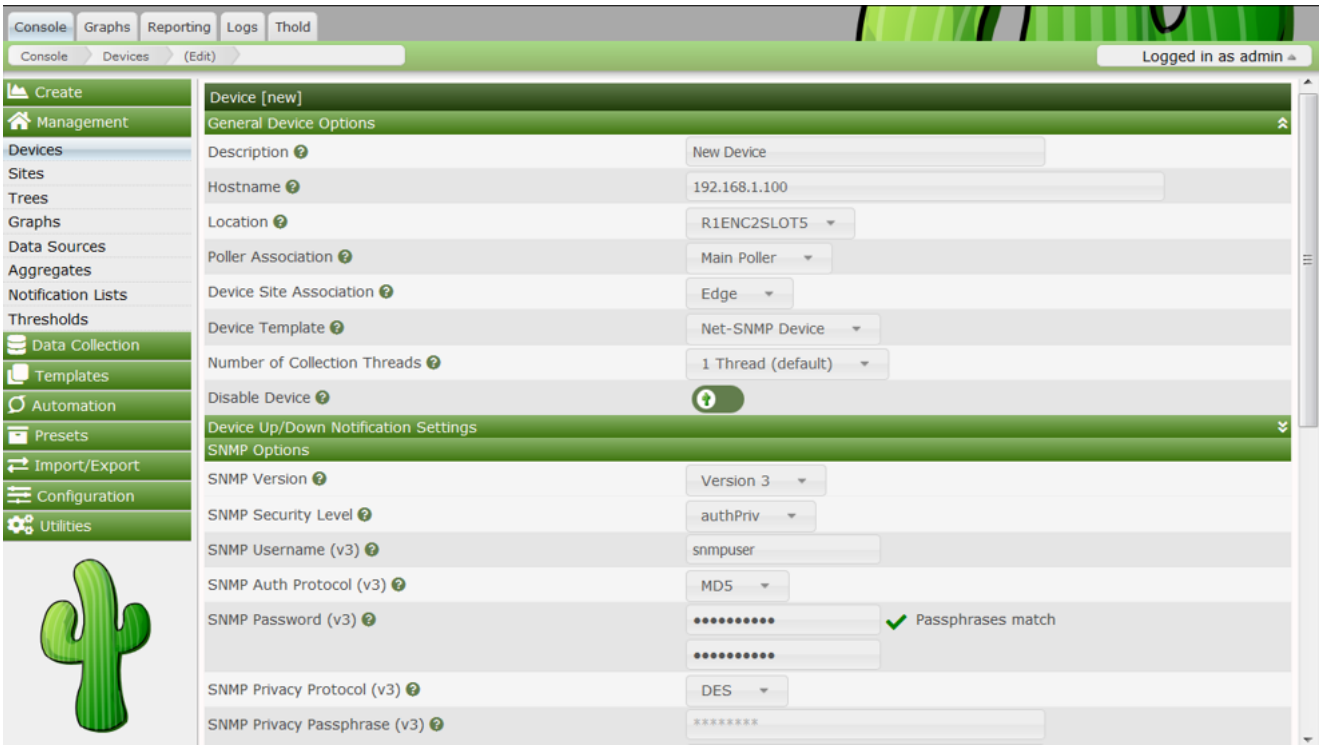
Copyright (c) 2004-2026 The Cacti Group

How to Graph Your Network

Create a device, select its measurements, and verify collection before adding its graphs to a tree. The manual workflow below is useful for a first device. Automation can perform these tasks for larger sets of devices once its discovery and graph rules are configured.

Creating a Device

Open **Console > Management > Devices** and click **Add**. Enter a description and address, choose the responsible data collector, and select a suitable Device Template. The template supplies associated graph templates and data queries; it does not restrict you to those defaults.



Device Field Definitions

- **Description** identifies the device in lists and default graph titles.
- **Hostname** is the device's address or resolvable DNS name.
- **Device Template** associates collection and graph definitions.
- **Poller Association** determines which collector reaches the device.
- **Site** groups devices by location.
- **Notes** records local information about the device.
- **Disable Device** stops collection while retaining its configuration.

Availability/Reachability Options

Choose a downed-device detection method appropriate to the device. Cacti can use SNMP, ping, both, either, or no availability test. A failed availability check can prevent measurements from being collected even when another protocol is reachable.

For ping checks, select the method, port where applicable, timeout, and retry count. ICMP permissions depend on the poller engine and operating system. Test connectivity from the collector assigned to the device.

SNMP Options

Configure the SNMP version and credentials accepted by the device:

- **SNMPv3** supports authentication and privacy. Prefer `authPriv` with mutually supported SHA and AES protocols where the device supports them.
- **SNMPv2c** supports 64-bit counters, but its community is not encrypted. Restrict access to trusted collectors and networks.
- **SNMPv1** is for legacy devices and does not support 64-bit counters.

Set the SNMP port, timeout, retries, and maximum OIDs per request. Start with values the device supports; increase request sizes only after verifying that responses are complete and collection remains reliable.

Security Options for SNMP V3

Choose the security level, username, authentication protocol and passphrase, privacy protocol and passphrase, and context or engine ID when required. The protocols offered depend on Cacti settings and the installed SNMP builds. The development defaults use SHA authentication and AES privacy. MD5 and DES are legacy algorithms and can be hidden by policy or unavailable in the build.

SNMP Retrieval Types

Cacti's PHP paths can use PHP's SNMP extension or external Net-SNMP tools. The extension supports SNMPv1, v2c, and v3; Cacti can select external tools for protocol combinations unavailable in the extension. Spine uses Net-SNMP through its C library. Verify the actual PHP, Net-SNMP, and Spine builds rather than assuming all offered algorithms are available on every platform.

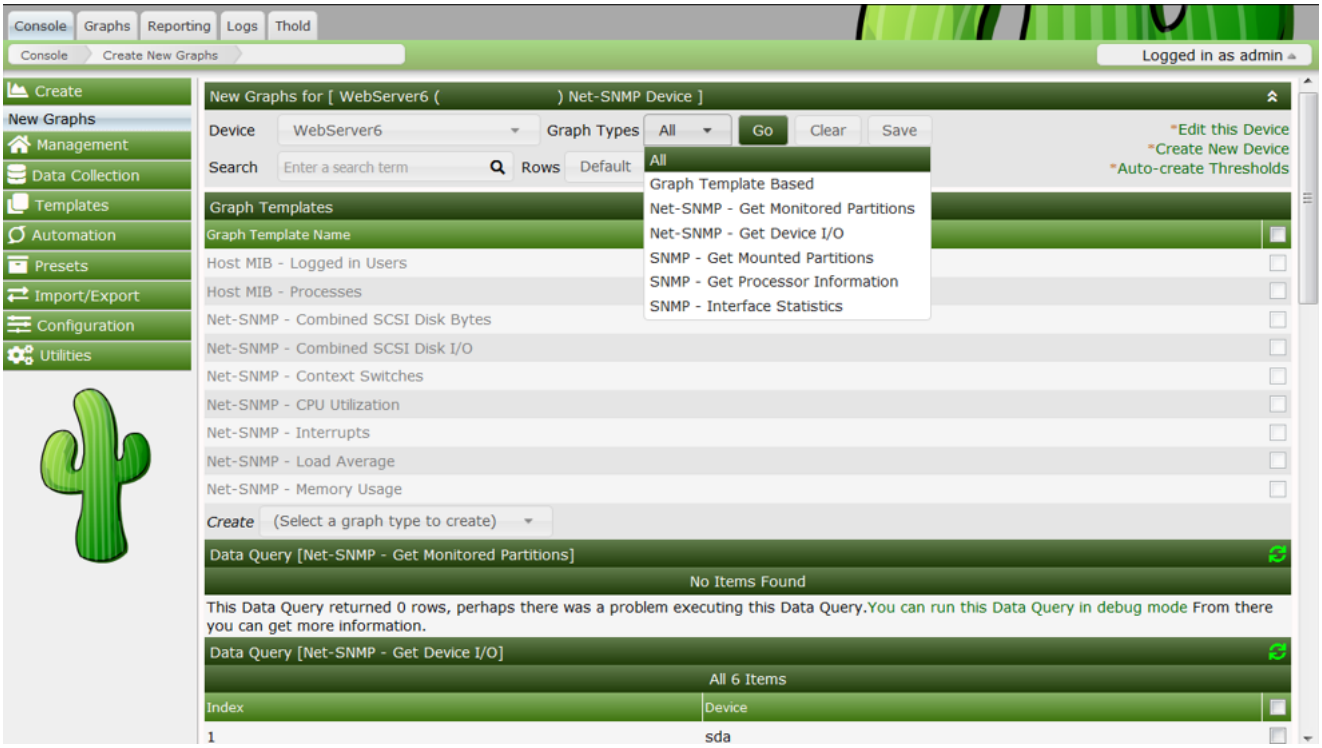
Verify device discovery

Save the device and check the SNMP information and any error shown on the edit page. Review **Associated Data Queries** and **Associated Graph Templates**. If a query has no rows, run its verbose query to check connectivity, OIDs, indexes, and credentials before creating graphs.

For interface traffic, prefer the 64-bit counter graph type when the device and selected SNMP version support it. See Data Queries for query-to-template mappings and Device Management for other availability and collection settings.

Creating the Graphs

Open **Console > Create > New Graphs** or use **Create Graphs for this Device** from the device page. Select the device, then choose the graph templates or query rows to graph. For indexed queries, choose the appropriate graph type before clicking **Create**.



Fill in any per-graph values requested by the templates and complete creation. Creating a graph defines collection; it does not immediately provide historical measurements. Allow scheduled polling to produce enough valid samples for the selected data-source type.

Verify and publish

1. Check the device's status and the Cacti log after polling.
2. Open the graph under **Console > Management > Graphs**. For blank graphs, inspect Data Source Debugging and the RRD last update.
3. Add the graph or device to a Tree, publish the tree, and grant the intended users access.
4. Confirm the result in Viewing Graphs with an account that has the intended permissions.

Copyright (c) 2004-2026 The Cacti Group

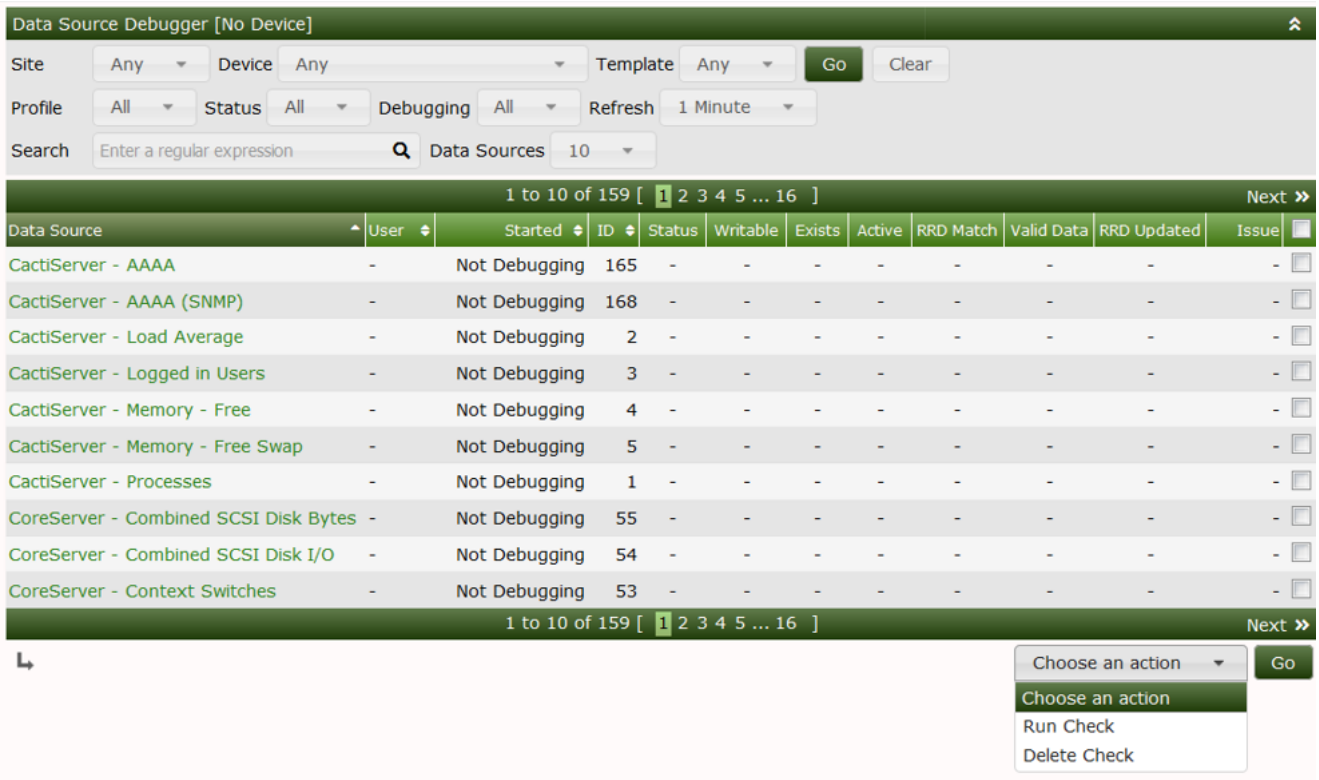
Data Source Debugging

If you are having issues with blank graphs and have done the basic checks such as permissions the Data source debug tool can help troubleshoot what information is coming into Cacti.

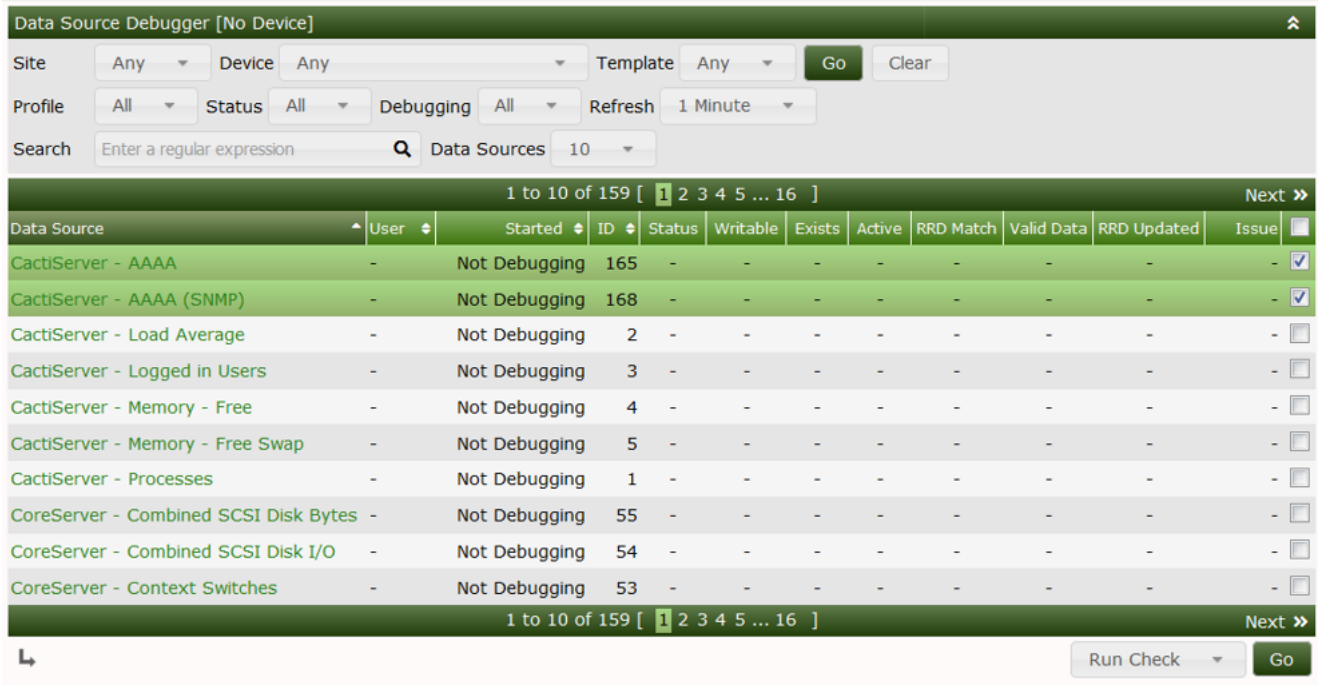
The debug tool will check the data source, graphs and permissions along incoming data and graph settings, such as step count, to ensure the definition matches the profile of the data source amongst other things.

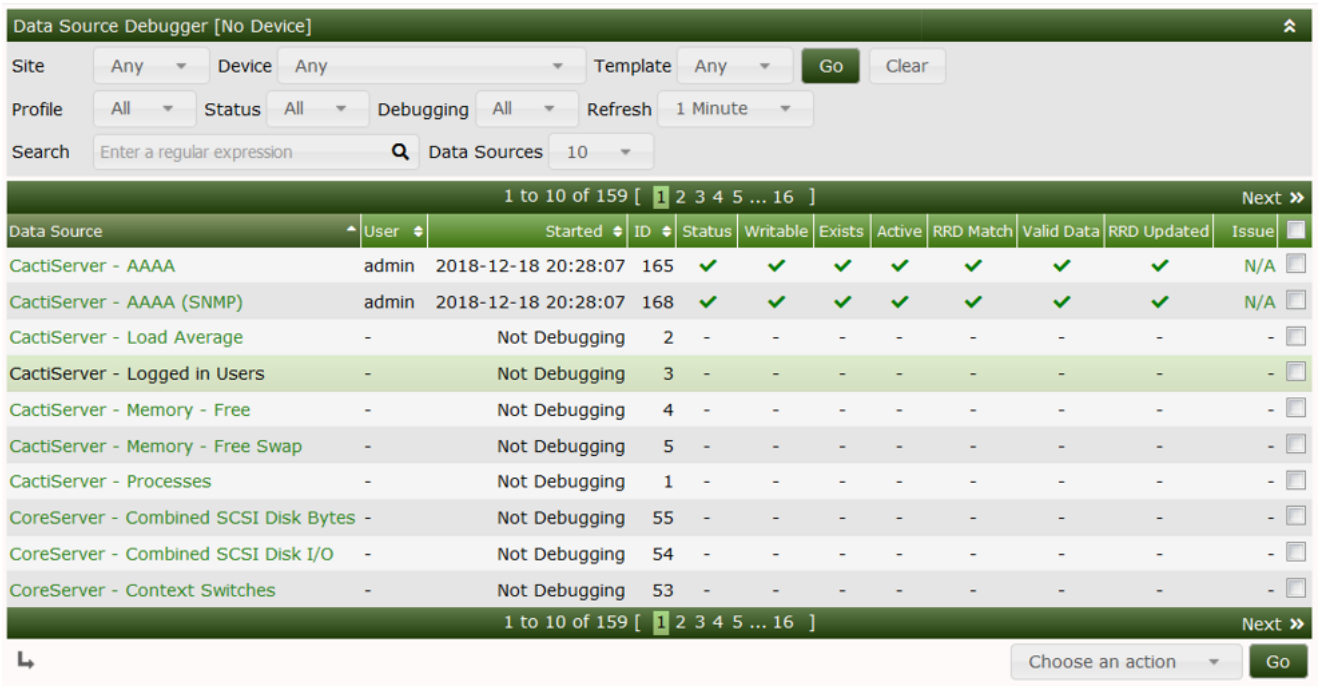
The tool allows you to run a debug on one or many data source. To request debug information, tick all applicable data sources you require information on, then select 'Run Check' in the actions drop down list. This will put all of the selected data sources into debug mode.

For a single data source, you can click the data source name which will also put it into debug mode.



You can see the status of each data source along with all of the checks run against it. Successful checks show a green checkmark, whilst failed checks shows a red X. Depending on the issue a recommendation will be provided for a failed check.





A summary of the checks as well as the properties of the data source can be displayed by clicking on the data source name. This is useful for a single page view of the data source and graph details.

Data Source Debugger		
Check	Value	Results
RRDfile Owner	apache:apache	-
Website runs as	apache	
Poller runs as	apache	
Is RRA Folder writeable by poller?	/var/www/html/cacti/rra	✔
Is RRDfile writeable by poller?	/var/www/html/cacti/rra/cactiserver_output_165.rrd	✔
Does the RRDfile Exist?	1	✔
Is the Data Source set as Active?	on	✔
Did the poller receive valid data?	output: -2.0	
Was the RRDfile updated?		Yes
First Check TimeStamp	1545182941	-
Second Check TimeStamp	1545183002	-
Were we able to convert the title?	CactiServer - AAAA	✔
Does the RRA Profile match the RRDfile structure?		✔
Issue		-

This page also shows what the last values where when last polled. This is also useful for troubleshooting as if the data comes in out of range of your Data Source template, or Graph template, this could point you into the right direction.

If the Web Server has write access to Cacti's RRA folder, if the Data Debug finds and issue that it knows how to correct. It will offer you a link to make that correction.

> **NOTE**: Devices that are down for long periods of time will report as having errors since the RRDfiles associated with them will not be updated as expected for with an available Device.

Copyright (c) 2004-2026 The Cacti Group

Debugging

Use this procedure when graphs show gaps, unknown values (`NaN`), or unexpected zeros. Start with one affected device and data source, and identify the failing stage: device or script, collector, database buffering, RRD storage, or graph rendering. The commands and menu labels follow Cacti development (1.3).

Use Data Source Debugging for the guided checks. Record the device, data-source and collector IDs, the expected polling interval, and the last successful sample. A down device or an idle counter can legitimately produce a different result from a failed collector.

Check Cacti Log File

Read the configured Cacti log, including the affected remote collector's log. The log path can be outside the application directory; the FreeBSD package normally uses `/var/log/cacti`.

Look for timeouts, authentication failures, script errors, database errors and poller overruns around the missing samples. Preserve the original log. Enable additional logging only for the affected device where supported, or briefly for a controlled polling cycle, then restore the previous logging level. Redact credentials and sensitive device details before sharing diagnostics.

An SNMP timeout does not establish that bulk requests are the cause. Check reachability, SNMP version, credentials, access controls and response time first. If the agent fails larger requests, adjust that device's request limits and retest rather than lowering global limits for every device.

Check Basic Data Gathering

Run a data-input script as the actual poller account, using the configured interpreter, arguments and environment. Compare its output with the expected field names and numeric values in the [Data Input Method](Data-Input- Methods.md). For a script-server input, use the PHP Script Server procedure; executing a function-based script directly is not an equivalent test.

For SNMP, query the exact OID with the same version, context and access settings as Cacti. Configure secrets through the supported protected Net-SNMP configuration rather than pasting real credentials into shared commands or logs. Check both the device response and Cacti's configured timeout/retries. A successful ping alone does not confirm that SNMP is available.

Check Cacti's poller

Verify the responsible Data Collector, its selected polling engine, and **Data Collection Enabled**. Check the actual launcher and **Cron/Daemon Interval** as well as **Poller Interval**. FreeBSD uses cron or configured rc.d services; Linux systemd commands do not apply there.

If a manual collector test is needed, first pause the overlapping scheduled work and wait for existing workers to finish. Restrict the test to the affected device. The development PHP collector uses named ID arguments, as shown by its command help:

```
php -q cmd.php --first=42 --last=42 --debug
```

Replace `42` with the device ID and run from the Cacti directory as the poller account. This collects data; it is not a read-only inspection. Restore the original schedule after the test. For Spine, use the installed binary's `--help` and Spine instructions for its version and configuration.

Check MySQL updating

Check the collector log for failed database writes, connection errors or replication problems. Inspect the Poller Cache and confirm that the affected data source is assigned to the expected collector.

Cacti can buffer samples for later processing. With Boost or an RRD cache enabled, collection and disk updates can occur at different times. Check the relevant worker and pending-data state before concluding that a missing direct `rrdtool update` log entry means collection failed.

Do not replay captured SQL writes against the production database as a debug step; they can duplicate or overwrite application state.

Check RRDfile updating

Use the RRD path shown by the data source and its configured collector. Do not assume it is under the web application's `rra` directory. On a default FreeBSD package installation, RRDs are under `/var/db/cacti/rra`.

Check the file's timestamps, data-source names and last update:

```
rrdtool info /path/to/device.rrd
rrdtool lastupdate /path/to/device.rrd
```

Use the configured RRDtool binary and, when applicable, the configured RRD cache connection. Account for pending cache writes before comparing disk timestamps. Do not issue manual production `rrdtool update` commands to fill a gap.

Check RRDfile ownership

The poller or RRD-writing service must be able to traverse the parent directories and write the RRD file. The graph-rendering process needs the access required by the configured storage/cache arrangement. Check those actual service identities rather than assuming they are the same user.

Inspect the affected file and its parent directories. Correct the specific ownership or permissions that differ from the deployment's policy. Avoid blanket ownership changes to application code, credentials, or every RRD file. See the FreeBSD installation guidance for package defaults.

Check RRDfile numbers

Read recent stored values and metadata:

```
rrdtool fetch /path/to/device.rrd AVERAGE --start -1h
rrdtool info /path/to/device.rrd
```

Check data-source type, minimum, maximum, step and heartbeat against the Data Source Template and Data Profile. An out-of-range value, counter reset or gap beyond the heartbeat can produce unknown data even when collection succeeds.

Changing a template does not necessarily rewrite existing RRD definitions. Plan any RRD repair or tuning against the actual files, preserve a consistent backup, and verify the result before applying it to other data sources. Increasing heartbeat can permit interpolation; it does not recover observed measurements that were never collected.

Check `rrdtool graph` statement

In Graph Management, open the affected graph and enable **Graph Debug Mode**. Inspect the command and RRDtool response. Check the `DEF` paths, data-source names, consolidation functions, requested time range and graph permissions.

If reproducing the command as the web account, preserve its binary paths and environment. Replace a `graph -` or `graphv -` output destination with a writable temporary image path so binary image output does not obscure diagnostics.

Duplicate Poller Entries

Inspect Cacti-specific cron entries, service configuration and running workers on the responsible collector. A package may have supplied a schedule in addition to one added manually. Keep one intended launcher; do not disable the system-wide cron service to stop a single Cacti job.

After resolving duplicate launches, verify a complete polling cycle and fresh RRD samples. Preserve the original service and scheduling configuration for rollback. See Settings - Poller.

Not NaN, but 0 (zero) values

Distinguish a valid zero from a script failure converted to zero. Run the input under the poller account, check its exit status and output, and verify external command paths. A successful interactive command as root does not establish that the collector has the same permissions or environment.

For tools that require elevated access, use a narrowly scoped mechanism supported by that tool or its package. Do not make a script or general-purpose interpreter setuid as a troubleshooting shortcut.

Copyright (c) 2004-2026 The Cacti Group

Hardening a Cacti Installation

This page covers OS-level hardening controls for a production Cacti host. The goal is defense in depth: even if a vulnerability in Cacti's PHP code is exploited, the attacker's reach should be limited.

Why These Controls Matter

A recurring class of vulnerability in Cacti involves writing an arbitrary file into the `resource/` or `scripts/` tree and then reaching that file over the web. The package-import advisory (GHSA-7cmj-g5qc-pj88, CVE-2024-25641) is a concrete example: a crafted package archive could write `resource/test.php` and execute it as a web request. The controls below close or limit that path at the OS and web-server layers, independent of the PHP fix.

---

1. File Permissions

Cacti writes to three paths at runtime. Everything else should be owned by root (or a non-web user) and not writable by the web process.

Path	Purpose	Recommended mode
`rra/`	RRDtool data files	`www-data:www-data 750`
`log/`	Cacti log files	`www-data:www-data 750`
`cache/`	Boost / poller cache	`www-data:www-data 750`
`resource/`	XML templates	`root:root 755` (read-only for web)
`scripts/`	Data-source scripts	`root:root 755` (read-only for web)
`lib/`, `include/`	PHP libraries	`root:root 755`

Apply once after installation:

```
# Writable paths (web user only)
chown -R www-data:www-data /var/www/html/cacti/rra \
                           /var/www/html/cacti/log \
                           /var/www/html/cacti/cache
chmod -R 750              /var/www/html/cacti/rra \
                           /var/www/html/cacti/log \
                           /var/www/html/cacti/cache

# Everything else read-only for the web user
chown -R root:root         /var/www/html/cacti
chmod -R 755               /var/www/html/cacti
# Re-apply writable exceptions after the recursive chmod
chown -R www-data:www-data /var/www/html/cacti/rra \
                           /var/www/html/cacti/log \
                           /var/www/html/cacti/cache

# Protect the database credentials in config.php.  The recursive 755 above
# leaves it world-readable; restrict it to root and the web user.
chown root:www-data /var/www/html/cacti/include/config.php
chmod 640           /var/www/html/cacti/include/config.php
```

> **Note:** The `scripts/` and `resource/` paths may be written during template import or package import by a privileged CLI process running as root, not by the web process. If your workflow requires the web process to write there, use a strict code-review policy for any import and consider the compat profile instead.

---

2. Web Server — Block Direct Access to Scripts and Resources

PHP files inside `scripts/` and `resource/` must not be reachable via HTTP. These directories hold data-source scripts and XML templates; a web request to a `.php` file written there is the direct exploitation path.

Apache

`<DirectoryMatch>` is only valid in server config or `<VirtualHost>` context, not `.htaccess`. Add this inside your Cacti `<VirtualHost>`:

```
# Deny direct HTTP access to the script and resource trees.
# A PHP file written here via import cannot be reached over the web.
<DirectoryMatch "^/var/www/html/cacti/(scripts|resource)/">
    <FilesMatch "\.php$">
        Require all denied
    </FilesMatch>
</DirectoryMatch>
```

If you only have access to `.htaccess` (e.g. shared hosting), use a `FilesMatch` block inside each directory instead:

```
# scripts/.htaccess and resource/.htaccess
<FilesMatch "\.php$">
    Require all denied
</FilesMatch>
```

Or, more broadly, deny all non-XML/non-script files from direct access:

```
<Directory "/var/www/html/cacti/scripts">
    Require all denied
</Directory>

<Directory "/var/www/html/cacti/resource">
    <FilesMatch "\.php$">
        Require all denied
    </FilesMatch>
</Directory>
```

Nginx

Add inside your `server {}` block:

```
# Block PHP execution in scripts/ and resource/
location ~ ^/cacti/(scripts|resource)/.*\.php$ {
    return 403;
}
```

---

3. PHP-FPM Pool Hardening

PHP itself warns that php-fpm pools are not a true security boundary, but they provide useful knobs. Create a dedicated pool for Cacti (`/etc/php/8.x/fpm/pool.d/cacti.conf`):

```
[cacti]
user  = www-data
group = www-data

listen = /run/php/php-cacti.sock
listen.owner = www-data
listen.group = www-data
```

```
; Drop all inherited environment variables — pass only what Cacti needs.
clear_env = yes

; Only execute .php files — blocks .phtml, .php3, .phar, etc.
security.limit_extensions = .php

; Disable dangerous functions at the pool level (not overridable by ini_set).
; Cacti's web process shells out to RRDtool, fping and helper binaries via
; exec, shell_exec, system, popen and proc_open (see lib/rrd.php, lib/ping.php,
; lib/spikekill.php), so disabling those breaks graph rendering and diagnostics.
; Only passthru and pcntl_exec are unused by the web tier and safe to disable.
; For tighter control, constrain the process with open_basedir and a hardened
; PATH rather than removing the shell-exec family Cacti depends on.
php_admin_value[disable_functions] = passthru,pcntl_exec

; Restrict file operations to the Cacti tree.
; NOTE: open_basedir disables the PHP realpath cache.  Measure the
; performance impact before enabling in production.  It is an extra
; safety net, not a full security boundary (PHP documentation).
; php_admin_value[open_basedir] = /var/www/html/cacti:/tmp

; Suppress PHP version disclosure.
php_admin_flag[expose_php] = Off
```

Reload php-fpm after changes:

```
systemctl reload php8.x-fpm   # Debian/Ubuntu
systemctl reload php-fpm       # RHEL/Rocky/Alma
```

---

4. SELinux (RHEL / AlmaLinux / Rocky Linux)

SELinux ships enforcing on RHEL-family systems. These commands apply the correct file contexts so Apache can serve Cacti while limiting what the web process can write.

File Contexts

```
CACTI=/var/www/html/cacti

# Read-only web content (Apache can read, not write)
semanage fcontext -a -t httpd_sys_content_t      "${CACTI}(/.*)?"
semanage fcontext -a -t httpd_sys_content_t      "${CACTI}/scripts(/.*)?"
semanage fcontext -a -t httpd_sys_content_t      "${CACTI}/resource(/.*)?"

# Read-write content for runtime paths only
semanage fcontext -a -t httpd_sys_rw_content_t   "${CACTI}/rra(/.*)?"
semanage fcontext -a -t httpd_sys_rw_content_t   "${CACTI}/log(/.*)?"
semanage fcontext -a -t httpd_sys_rw_content_t   "${CACTI}/cache(/.*)?"

# Apply the contexts
restorecon -Rv "${CACTI}"
```

Database Network Access

If your database server is reached over TCP (not a local Unix socket), enable the SELinux boolean that permits it:

```
setsebool -P httpd_can_network_connect_db on
```

Poller Systemd Drop-in

The Cacti spine poller runs as a separate daemon and may need to connect outward for SNMP. Do not force it into the `httpd_t` domain: that confines spine to Apache's policy rather than freeing it, and outbound access in `httpd_t` is still gated by booleans. If SNMP polling is blocked by SELinux, enable the relevant boolean instead and leave the daemon in its own domain:

```
setsebool -P httpd_can_network_connect on
```

Run `systemctl daemon-reload && systemctl restart cactid` after changes.

> **Verify:** After applying contexts, confirm Apache can still serve pages and the poller can poll. Use `audit2why` on any AVC denials in `/var/log/audit/audit.log`.

---

5. AppArmor (Debian / Ubuntu)

Ubuntu Server ships AppArmor enforcing by default. Create a profile for the Cacti php-fpm pool at `/etc/apparmor.d/cacti-fpm`:

```
#include <tunables/global>

profile cacti-fpm /usr/sbin/php-fpm* {
    #include <abstractions/base>
    #include <abstractions/php>
    #include <abstractions/mysql>

    # Cacti tree — read-only by default
    /var/www/html/cacti/           r,
    /var/www/html/cacti/**         r,

    # Allow execution of Cacti PHP files
    /var/www/html/cacti/*.php      r,
    /var/www/html/cacti/**/*.php   r,

    # Runtime write paths
    /var/www/html/cacti/rra/**     rw,
    /var/www/html/cacti/log/**     rw,
    /var/www/html/cacti/cache/**   rw,

    # Helper binaries Cacti shells out to.  AppArmor is default-deny once
    # file rules are present, so these need explicit execute (ix) or graphs
    # and polling break under enforce mode.
    /usr/bin/rrdtool               ix,
    /usr/bin/fping                 ix,
    /usr/bin/ping                  ix,
    /usr/bin/snmpget               ix,
    /usr/bin/snmpbulkwalk          ix,
    /usr/bin/php                   ix,

    # Temp files
    /tmp/**                        rw,
    /var/tmp/**                    rw,

    # Deny write to scripts/ and resource/
    deny /var/www/html/cacti/scripts/**  w,
    deny /var/www/html/cacti/resource/** w,
}
```

Load and enforce the profile:

```
apparmor_parser -r /etc/apparmor.d/cacti-fpm
aa-enforce /etc/apparmor.d/cacti-fpm
```

Attach the Profile to the php-fpm Service

Add a drop-in at `/etc/systemd/system/php8.x-fpm.service.d/apparmor.conf`:

```
[Service]
AppArmorProfile=cacti-fpm
```

Reload and restart:

```
systemctl daemon-reload
systemctl restart php8.x-fpm
```

Use complain mode (`aa-complain`) first to test without blocking, then switch to enforce once logs show no legitimate denials:

```
# Test mode
aa-complain /etc/apparmor.d/cacti-fpm

# Enforce when ready
aa-enforce  /etc/apparmor.d/cacti-fpm
```

---

6. Verification Checklist

After applying all controls, confirm:

- [ ] `curl -s http://localhost/cacti/scripts/ss_webseer.php` returns 403
- [ ] `curl -s http://localhost/cacti/resource/` returns 403 for any `.php` inside
- [ ] The Cacti web UI loads and graphs render (rra/ is writable)
- [ ] The poller runs without errors (`tail -f /var/www/html/cacti/log/cacti.log`)
- [ ] On RHEL: no AVC denials in `ausearch -m avc -ts recent`
- [ ] On Debian/Ubuntu: no AppArmor denials in `journalctl -k | grep apparmor`

See Also

- Installing Under Ubuntu/Debian
- Installing Under Enterprise Linux (LAMP)
- Standards Security

Copyright (c) 2004-2026 The Cacti Group

Spine

Use only the supported operating systems and maintained dependencies compatible with this Cacti branch.

Spine is Cacti's C-based, multithreaded data collector. It replaces `cmd.php` as the collection engine; `poller.php` remains the scheduling entry point. Actual throughput depends on device latency, scripts, concurrency, database capacity, and the collection interval. Measure your installation before tuning.

Spine Architecture and Operating Principles

Spine reads polling definitions from the Cacti database, collects measurements using SNMP and scripts, and writes results to poller output tables. Cacti's poller or Boost processing updates the RRD files. Spine does not directly write historical RRD files or render graphs.

Spine can run several processes and multiple threads per process. It also maintains PHP script-server processes for methods configured to use `script_server.php`, avoiding a separate PHP startup for each such call.

Installing Spine

Use the Spine release corresponding to your Cacti release. For development checkouts, use compatible branches; do not mix a development poller with a maintenance installation without verifying compatibility.

Install a C compiler, make, Autoconf, Automake, Libtool, and the development headers and libraries required by that Spine version, including Net-SNMP and MySQL or MariaDB client libraries. Git checkouts also need the bootstrap tools. Use the package names and build repository appropriate to your distribution.

Download source from Cacti Spine releases or use your distribution's compatible package. From the extracted source:

```
./bootstrap
./configure --prefix=/usr/local/spine
make
sudo make install
```

Inspect `./configure --help` for library locations and optional build features. Package installations can use different paths and permission policies.

ICMP permissions

Run the poller as its dedicated service account. ICMP availability checks need socket permissions appropriate to the operating system and Spine version. Possible mechanisms include permitted ping sockets, a narrowly scoped raw socket capability, or the release's documented setuid installation.

The development branch can use unprivileged ICMP where the operating system permits it. When started setuid root, it opens the required sockets and drops root permanently before reading configuration. Older release behavior differs. Do not assume Spine ships setuid or repeatedly regains root while polling.

Follow the installed release's instructions and local security policy. Keep the executable owned by an administrative account and unwritable by the poller account. Test the chosen availability method as the actual poller user. SNMP collection does not itself require running the entire poller as root.

Configuring `spine.conf`

Create the configuration file from the installed template if one is not already provided by the package:

```
sudo cp /usr/local/spine/etc/spine.conf.dist /usr/local/spine/etc/spine.conf
```

Set the database host, database name, user, password, and port to match the collector's database. Use that release's `spine.conf.dist` as the reference. For example:

```
DB_Host         localhost
DB_Database     cacti
DB_User         cactiuser
DB_Pass         REPLACE_WITH_DATABASE_PASSWORD
DB_Port         3306
```

Use the `DB_UseSSL` and `DB_SSL_*` options when configuring the local database connection for TLS. Remote collectors also have `RDB_*` settings for the main database. Do not confuse local `DB_*` with remote `RDB_*` settings.

Restrict the configuration file to the accounts that need to read it, because it contains database credentials. One Linux example, with a poller in the `www-data` group, is:

```
sudo chown root:www-data /usr/local/spine/etc/spine.conf
sudo chmod 0640 /usr/local/spine/etc/spine.conf
```

Substitute the actual poller group. Do not change application code ownership to make a credential file readable.

Testing Spine from the Command Line

Use the poller account to run a database read-only test. The `-R` option suppresses database writes, but still polls devices and executes configured scripts. It is not a guarantee that those scripts have no side effects.

```
/usr/local/spine/bin/spine -R -V 3 -S
```

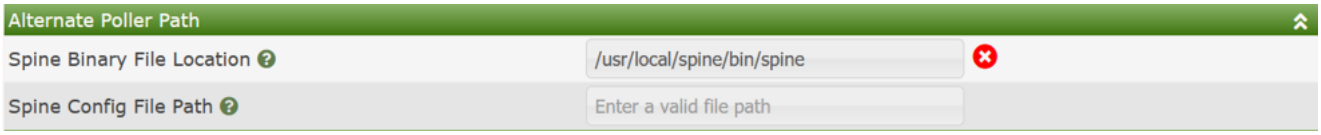
To restrict the test to one device, substitute its actual ID for `DEVICE_ID`:

```
/usr/local/spine/bin/spine -R -f DEVICE_ID -l DEVICE_ID -V 5
```

Confirm the configuration path, database connection, device availability, and returned values. Verbose output can expose sensitive operational information; redact it before sharing. Use the installed binary's `--help` for its options.

Activating Spine in Cacti

1. Open **Console > Configuration > Settings > Paths** and set **Spine Binary File Location** to the installed executable.
2. Set **Spine Config File Path** when using a non-default configuration path.
3. Open **Settings > Poller**, choose **spine** as the **Poller Type**, and save.
4. Leave the existing `poller.php` scheduler in place. Check the next scheduled cycle for errors and confirm that graph timestamps advance.



Tuning Spine for Optimal Performance

Change one setting at a time and compare elapsed polling time, missed samples, CPU use, database connections, and errors. More concurrency can overload a collector, database, or device.

- **Settings > Poller** contains global script-server counts and script timeout settings. Allow expected script execution time within the overall polling budget.
- **Data Collection > Data Collectors** sets processes and threads for each collector. Size them for that collector's workload and database capacity.
- **Management > Devices** provides device threads and maximum OIDs per request. Larger SNMP requests can reduce round trips, but can also exceed agent limits or fragment across the network.

There is no universal process-per-core ratio, maximum useful thread count, or OIDs-per-request value. Start with the release defaults and measured results.

Database Connection Sizing

Budget database connections across all collectors, poller processes, threads, script servers, web requests, and maintenance jobs. Remote collection can require connections to both local and main databases. Use Cacti's technical support information and observed peak connections to size `max_connections`.

A connection failure can indicate credentials, DNS, routing, TLS, account-host restrictions, or exhausted capacity. Check the database error before increasing limits, and account for the memory cost of additional connections.

Security Policies

SELinux or AppArmor can restrict execution, network connections, or configuration file access. Inspect policy denials and apply rules appropriate to the poller account and executable. Do not disable enforcement to diagnose a path or credential error. See Hardening.

Troubleshooting Spine

- **Configuration unreadable**: verify the configured path and read access as the poller user.
- **ICMP failure**: check the selected ping method, socket permissions, operating-system policy, and firewall. Do not assume SNMP must also fail.
- **Database failure**: verify the correct local or remote connection settings and inspect the database's error and connection count.
- **Missing samples**: inspect script output, timeouts, SNMP responses, and the configured collection interval.

A `SYSTEM STATS` log line containing `Method:spine` confirms the selected engine, not successful collection of every item. Check errors and actual RRD updates as well. See Data Source Debugging and The Cacti Log.

Copyright (c) 2004-2026 The Cacti Group

Upgrading Cacti Under UNIX Compatible Operating Systems

Use only the supported operating systems and maintained dependencies compatible with this Cacti branch.

This procedure covers a source installation upgrading to Cacti development (1.3). For a package-managed installation, use the package maintainer's upgrade procedure and configuration-file handling. See the separate guides for Windows and FreeBSD.

Review the release notes and requirements first. Schedule time to back up and restore the installation. Test the upgrade on an isolated copy of the database and files before changing production. Keep dependency upgrades separate unless a prerequisite requires them.

Stop writers before backing up

Turn off **Data Collection Enabled** under the poller settings. Also suspend the actual scheduler on each affected collector: disable only the Cacti cron entry, or stop its installed `cactid` service. Preserve the scheduler settings so they can be restored later. Do not disable all system cron jobs.

Wait for existing poller, Spine, script-server, Boost and other background jobs to finish. Prevent web requests and plugin jobs from changing the installation during the backup and upgrade. The application polling switch alone does not stop every running process or administrative writer.

If collectors use `rrdcached`, drain its pending writes using the procedure for that deployment before taking the RRD snapshot. Keep writers stopped until the database and file backups are both complete. Coordinate this on remote collectors as well as the main server.

Back up the database and complete installation

Record the installed Cacti, Spine, PHP and database versions, installation paths, service accounts, scheduler entries and remote-collector configuration. Store backups outside the web root with access restricted to administrators.

Back up the Cacti database with a client compatible with the database server. For example, with the application stopped and a backup account with the required privileges:

```
umask 077
BACKUP_DIR=/srv/backups/cacti-before-1.3-dev
mkdir "$BACKUP_DIR"
mysqldump --user=backup_user --password --lock-tables --quick \
  --routines --events --triggers cacti \
  > "$BACKUP_DIR/cacti.sql"
```

Adjust the database name, account and paths before running these examples. Use `mariadb-dump` instead of `mysqldump` where appropriate. Enter the password at the prompt. Stop if a command fails; an output file alone does not prove a successful dump. The account needs the privileges required by the installed dump client, including table locks for the example above.

`--single-transaction` provides a consistent snapshot for transactional tables; it does not make nontransactional tables consistent. Inspect the actual storage engines and choose the backup method accordingly. Keep all application writers stopped for the corresponding file snapshot. See the database backup documentation for the option and privilege details.

Back up the complete installation, including the RRD files. For an example installation at `/var/www/html/cacti`:

```
tar -czpf "$BACKUP_DIR/cacti-files.tar.gz" -C /var/www/html cacti
tar -tzf "$BACKUP_DIR/cacti-files.tar.gz"
```

Preserve `include/config.php`, plugins, custom scripts, query resources and other local configuration. Include CSRF secret material, including an alternate `$path_csrf_secret` from the configuration. Back up separately any RRD, configuration or resource paths outside the installation tree. A tar archive normally preserves a symlink rather than including its external target. Retain the permissions and ownership needed for restoration.

Save the database user/grant definitions and any connection certificates needed to recreate access. These are not included in a dump of the Cacti database alone. Include site or plugin SQL routines and events, as in the example, and account for their additional backup privileges.

Back up remote-collector databases and local files where they hold state. Verify dump completion and archive contents, then rehearse restoration on an isolated system. Excluding `*.rrd` would leave monitoring history outside the recovery set.

Check development dependencies

Development source requires PHP 8.3 or later in the PHP 8 series, with the extensions declared by the selected revision. Install locked dependencies in the staging directory with `composer install --no-dev`, then run `composer check-platform-reqs --no-dev`. Preserve this complete dependency tree when deploying the staged application. Do not run `composer update` against a live installation. A complete release bundle can already include dependencies; verify its platform checks instead of assuming that every archive is complete.

Install the staged files

Obtain the intended development revision and stage it outside the live installation. Record its commit and verify its provenance. If using a release archive instead, follow that release's requirements and verification steps. Keep the existing installation and its backup until the upgrade has been checked.

Copy the contents of the prepared staging directory into the existing installation, not the directory itself. For example:

```
STAGING_DIR=/var/tmp/cacti-1.3-dev
CACTI_DIR=/var/www/html/cacti
cp -a "$STAGING_DIR"/. "$CACTI_DIR"/
```

Confirm that the new files are directly under the installation root, such as `$CACTI_DIR/poller.php`, rather than a nested `cacti-1.3-dev` directory. Preserve the existing `include/config.php` and review new options in `include/config.php.dist`. Reconcile local code changes and plugin compatibility with the new files; copying the release is not a substitute for reviewing those changes.

Check permissions using the accounts that actually run the web server, poller and maintenance jobs. Those accounts need not be identical. Give the poller the required access to RRD and log paths, and check configured cache, resource and script paths. Preserve access needed by installation and package imports. Do not make the entire application tree writable as a permissions workaround.

Run and verify the upgrade

Allow an administrator to open the Cacti URL and follow the database-upgrade workflow. Review errors before proceeding. The release also provides `cli/upgrade_database.php`; consult its `--help` output if using the CLI workflow. Do not manually advance the database version or force a starting version merely to bypass an upgrade check.

Confirm the installed version, database-upgrade result, credentials, paths, plugin state and collector configuration. Check remote collectors individually and follow the applicable synchronization procedure; do not assume that two polling cycles prove their files and databases have upgraded successfully.

Restore web access and the saved scheduler configuration only after the upgrade checks succeed, then re-enable collection. Use either the configured cron path or the daemon for a collector, not both. Verify logs, data freshness and representative RRD updates after normal polling resumes.

Recovery

If the upgrade fails, keep writers stopped while deciding whether to retry or restore. Replacing PHP files alone does not undo a database migration. Restore the matching pre-upgrade database, application files, RRD data and external configuration or secret material, including affected remote-collector state. Verify the restored installation before re-enabling its schedulers. Restoring an earlier backup discards changes collected after that backup.

Copyright (c) 2004-2026 The Cacti Group

Upgrading Cacti Under FreeBSD

Use only the supported operating systems and maintained dependencies compatible with this Cacti branch.

These instructions cover the FreeBSD upgrade procedure. This manual describes Cacti development (1.3), while the FreeBSD port currently supplies Cacti 1.2.31. Use the 1.2.x manual for release-specific requirements when upgrading the packaged release. Determine whether Cacti is maintained by `pkg`, by ports or from a source archive. Keep that installation method for the upgrade; do not overlay an archive onto package-managed files. Review the target Cacti release notes, requirements and package or port notes before proceeding. Do not make an unrelated operating-system, PHP or database major-version upgrade part of this procedure without a separate compatibility plan.

Pause collection and back up

1. Clear **Data Collection Enabled** under `Console > Configuration > Settings

   > Poller`. Disable the Cacti-specific cron entry or stop its configured poller service on every collector. Record what you change; do not stop the system-wide cron service merely to pause Cacti. Wait for pollers, Spine, script servers and other background writers to finish. Restrict web access to the upgrade administrator and pause plugin jobs that write to Cacti. Flush pending RRD-cache writes and confirm that the database and RRD files remain unchanged before taking the backup.

2. Use the dump client appropriate to the installed MySQL or MariaDB server. The following example requires an existing, protected backup directory and a database account with the server's required backup privileges. Replace the account, database name and filename for your installation. Use its actual database host, port and TLS options.

   ```
   mysqldump --user=YOUR_BACKUP_USER --password \
     --lock-tables --quick --routines --events --triggers \
     --databases cacti --result-file=/protected/backup/cacti-YYYYMMDD.sql
   ```

   The password is prompted. With all Cacti writers paused, this dump method uses table locks for the Cacti database, including nontransactional tables. A transaction-only dump does not cover those tables. Verify the command succeeded and test restoring the dump separately; a nonempty file alone is not proof of a complete backup. Use a new filename rather than overwriting a known good backup.

3. Back up the Cacti application files, RRDs and local configuration together with the database dump. Preserve ownership, permissions and hidden files. Include `include/config.php`, custom scripts, query resources, plugins, secret files, including any alternate `path_csrf_secret`, and configured paths outside the application tree. Record symbolic links and back up their targets separately. Also record the Spine configuration, cron/service settings, web-server settings, database grants, certificates/keys and installed versions. Package layouts may keep configuration and data outside `/usr/local/share/cacti`; inspect the installed paths rather than backing up that directory alone. Include `/var/db/cacti/rra` and `/var/log/cacti` when those are the configured package paths, and protect the backup directory from other users because the configuration contains credentials.

Upgrade using the existing installation method

For a binary package installation, review the proposed package transaction before applying it. The FreeBSD Handbook describes package upgrades. Run the following with appropriate administrator privileges:

Discover the installed flavor first:

```
pkg info -o -x '^cacti'
pkg update
```

For an installed `cacti-php84` package, for example:

```
pkg upgrade -n cacti-php84
pkg upgrade cacti-php84
pkg info -D cacti-php84
```

Replace `cacti-php84` with the installed package name. Keep the PHP flavor unless a separate migration is intended. If Spine is installed, review its upgrade and verify its version remains compatible with Cacti. See the FreeBSD installation section for package paths and the `net-mgmt/cacti-spine` port.

Check the proposed version and dependency changes before accepting the actual upgrade. A package repository normally supplies a release rather than development source. Stop if that is not your approved target and select an appropriate repository or installation method. Do not delete Cacti first or assume that upgrading one package cannot affect its dependencies.

For a ports-managed installation, follow the handbook's ports upgrade procedure and applicable `/usr/ports/UPDATING` entries. Use the existing ports management method; do not switch between ports and binary packages without reviewing their options and dependencies. A source installation instead follows Upgrading Cacti.

Preserve the working configuration and merge required changes from the new configuration example. Keep existing database credentials, collector IDs, paths and secrets. Review package-generated configuration changes and restore local resources only after checking compatibility. Recreate cache or data directories only where the installed configuration requires them, with access for the actual service account; do not assume a universal account or apply recursive ownership changes to the whole installation.

Check development dependencies

Development source requires PHP 8.3 or later in the PHP 8 series, with the extensions declared by the selected revision. Install locked dependencies in the staging directory with `composer install --no-dev`, then run `composer check-platform-reqs --no-dev`. Preserve this complete dependency tree when deploying the staged application. Do not run `composer update` against a live installation. A complete release bundle can already include dependencies; verify its platform checks instead of assuming that every archive is complete.

Complete the application upgrade

Open the existing Cacti URL as an administrator and complete its database upgrade. Review the installer and logs before resuming collection. Upgrade remote collectors' local files and configuration through their installation method, complete the applicable upgrade/synchronization, and verify their versions, connectivity and logs. Do not rely on a fixed number of poll cycles for completion. Check the matching Spine version on collectors using Spine.

After successful checks, re-enable **Data Collection Enabled** and restore the recorded Cacti cron entry or service. Verify a complete poll cycle, recent RRD samples and representative graphs on every collector. If recovery is needed, stop all writers again and restore the matching files, database, RRDs and configuration as one set; restoring old application files alone does not undo a database upgrade.

Copyright (c) 2004-2026 The Cacti Group